Politecnico di Milano

Numerical Analysis for Machine Learning Project

Trustworthy pneumonia detection in chest X-ray imaging through attention-guided deep learning

Student:
Emanuele De Simone
Matricola: 303143

Academic Year 2025 / 2026

Contents

1 Introduction **2**
1.1 Scope . . . 2
1.2 Acronyms . . . 2

2 Mathematical Models **3**
2.1 Spiking Neural Network . . . 3
2.1.1 Leaky integrate-and-fire neuron model . . . 3
2.2 Gated Recurrent Unit . . . 3

3 Neural Network Architecture **5**
3.0.1 CNN Block . . . 5
3.0.2 Flattening Block . . . 5
3.0.3 Linear Feature Projection Block . . . 5
3.0.4 BiGRU Block . . . 6
3.0.5 SNN Block . . . 6
3.0.6 Decision Head Block . . . 7

4 Technologies Used **8**

5 Experimental Setup **8**
5.1 Preprocessing . . . 8
5.2 Cross-validation and Training . . . 9
5.3 Testing . . . 9

6 Experimental Results **10**

7 Conclusion **15**

1 Introduction

1.1 Scope

The aim of this project is to replicate the results obtained in the scientific paper *Trustworthy pneumonia detection in chest X-ray imaging through attention-guided deep learning* by *Houmem Slimi, Ala Balti, Sabeur Abid and Mounir Sayadi* on Nature.
The goal of the authors is to propose an efficient alternative to the classical architectures used to classify images through neural networks, in particular, a neural network that classifies chest X-ray images in two classes: sane people and people with pneumonia.
The main idea of the paper is to use spiking neural networks, a class of neural models that performs efficiently on time-dependent data, because of their suitability with neuromorphic hardware and their capability of mimicking the processes of the brain, leading to energy-efficient computation. Therefore, the challenge is to find a way to somehow transform static X-ray images into dynamic data for the spiking neural network.
The purpose of this document is to briefly describe the **mathematical models** and the **architecture of the neural network** devised by the authors of the paper, to list the main assumptions and choices made to replicate some parts that the paper did not examine in depth or were found ambiguous and, finally, the main experimental findings of the newly implemented neural network.

1.2 Acronyms

Acronym	Definition
NN	Neural Network
CNN	Convolutional Neural Network
SNN	Spiking Neural Network
LIF	Leaky integrate-and-fire
GRU	Gated Recurrent Unit
RNN	Recurrent Neural Network
BiGRU	Bidirectional Gated Recurrent Unit

2 Mathematical Models

In this section, a description of the most relevant mathematical concepts used to implement the different blocks of the NN is given.

2.1 Spiking Neural Network

SNNs are a class of NNs that behave similarly to biological neurons, meaning that, unlike classical continuous activation functions such as ReLU or sigmoid, neurons will output binary signals ("spikes") in response to an accumulating input exceeding a certain threshold.
SNNs have been chosen because of their compatibility with neuromorphic hardware, a particular kind of hardware that, like SNNs, mimics the structure and functions of the brain, to improve efficiency both accuracy and energy-wise.

2.1.1 Leaky integrate-and-fire neuron model

LIF neurons are the ones used in the SNN layers and their behavior is the following:

- *Membrane Potential Update*:

$$V(t+1) = \beta V(t) + (1-\beta) I(t)$$

  where V(t) is the membrane potential, I(t) is the input of the neuron and beta is an hyperparameter called *decay factor*;

- *Spike Generation*:

$$\text{Spike}(t) = \begin{cases} 1 & \text{if } V(t) \geq V_{\text{thresh}} \\ 0 & \text{otherwise} \end{cases}$$

  where $V(t)$ is the membrane potential and V_{thresh} is the threshold above which the membrane potential is reset to V_{reset};

- *Reset Mechanism*:

$$V(t) = V_{\text{reset}} \quad \text{if Spike}(t) = 1$$

  where V(t) is the membrane potential and V_{reset} is the reset value;

- *Output*:

$$\text{Output} = \frac{1}{T} \sum_{t=1}^{T} \text{Spike}(t)$$

  so the output is the average over time of all the spikes "fired" in the time span given;

2.2 Gated Recurrent Unit

The goal of this block is to transform static images into dynamic data for the SNN. GRUs are a particular kind of RNN, so NNs that at each time instant somehow remember their previous output (called *hidden state*) retaining a sort of short-term memory, that use "gates" to decide what to memorize and what to forget at each instant. A gate is just a vector computed at each instant used to compute the new hidden state. Here a list of the most important concepts:

- *Update Gate*:
$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$
where W_z, U_z and b_z are the weights and bias, x_t is the current input and h_{t-1} is the hidden state of the previous instant;

- *Reset Gate*:
$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$
where W_r, U_r and b_r are the weights and bias, x_t is the current input and h_{t-1} is the hidden state of the previous instant;

- *Candidate Hidden State*:
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$
where W_h, U_h and b_h are the weights and bias, x_t is the current input and h_{t-1} is the hidden state of the previous instant. The current candidate hidden state depends on the current input and on the previous hidden state rescaled by the current reset gate (values between 0 and 1);

- *Hidden State*:
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$
finally, the current hidden state is s computed as a convex sum between the current candidate hidden state and the previous hidden state. The idea is that if the update gate is close to 1, the current state depends mostly on $\tilde{h}_t$, while if the update gate is close to 0 then h_t depends mostly on h_{t-1}.

In this case, a bidirectional GRU has been used, that is a GRU that reads input data concurrently in two different directions, forward and backward with respect to time. This means that the BiGRU reads data from instant 0 to the final instant End and at the same time from End to 0, producing two outputs that are then concatenated.

3 Neural Network Architecture

In this section, the chosen architecture is discussed in greater detail, focusing on the role of each block, and highlighting the differences from the reference paper. Figure 1 illustrates the high-level architecture of the proposed neural network.

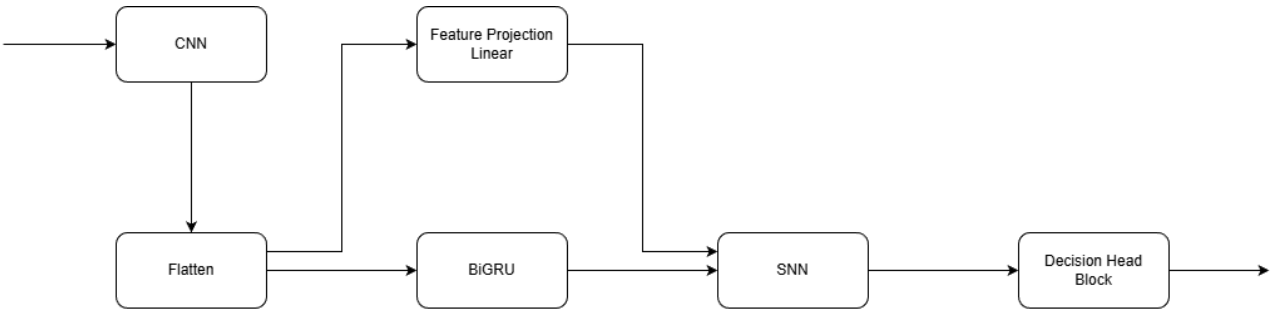


Figure 1: Architecture

3.0.1 CNN Block

The first unit of the whole NN is a convolutional neural network, whose purpose is to extract the spatial features of the images in input. The CNN has three layers, it takes as input a $3 \times 128 \times 128$ tensor (an input image with its three channels) and outputs a $128 \times 16 \times 16$ one. Here follows a table explaining precisely how each layer has been structured:

Layer	Input Dimension	Convolution	Output Dimension
1	$3 \times 128 \times 128$	`kernel_size=3, stride=1, padding=1`	$32 \times 64 \times 64$
2	$32 \times 64 \times 64$	`kernel_size=3, stride=1, padding=1`	$64 \times 32 \times 32$
3	$64 \times 32 \times 32$	`kernel_size=3, stride=1, padding=1`	$128 \times 16 \times 16$

Table 1: CNN Layers

Moreover, to each layer it has been applied batch normalization, a leaky ReLU activation function with slope equal to 0.1 and a max pooling with kernel size and stride both equal to 2 (the reason why the image dimensions are halved at each layer).

3.0.2 Flattening Block

This block purpose is just to flatten on a vector the output of the CNN, so it takes as input a $(128 \times 16 \times 16)$ tensor and outputs a vector of dimension equal to 32768.

3.0.3 Linear Feature Projection Block

The SNN block will receive in input both the output of the BiGRU and the one of the CNN, therefore, in order to grant a smooth transition between the CNN and the SNN, we introduce this block just to compress the flattened output of the CNN into a smaller vector through a NN of three layers without activation function (this is why we call it "linear"). Here a table explaining this block more in depth:

Layer	Input Dimension	Output Dimension
1	32768	128
2	128	64
3	64	256

Table 2: Linear Feature Projection

3.0.4 BiGRU Block

This block plays a critical role, since it transforms the spatial features of the image extracted by the CNN (and then reduced by the linear projection block) into a temporal sequence that will feed the SNN.

Parameter	Value
`num_steps`	25
`input_size`	32768
`hidden_size`	256
`num_layers`	2
`bidirectional`	True
`dropout`	0.3

Table 3: BiGRU Parameters

Here follows an explaination of the configuration parameters shown in the table above:

- `num_steps`: number of times the flattened output of the CNN is given as input to the BiGRU;
- `input_size`: length of the CNN flattened output;
- `hidden_size`: size of the hidden state of the BiGRU (h_t in the mathematical models described previously);
- `num_layers`: number of layers of the BiGRU unit;
- `bidirectional = True`: a flag set to true to use the bidirectional model;
- `dropout`: hyperparameter turning off some neurons during training to favor generalization in spite of overfitting.

As explained in the mathematical models section, the BiGRU outputs are two vectors whose dimension is the same of the hidden state (so 256) that are concatenated, for a total output vector of dimension equal to 512.

3.0.5 SNN Block

The SNN block iterates over the 25 time instants artificially generated by the BiGRU to generate spikes once a certain threshold is reached. In the end, the output is obtained by making the average of all the spikes fired over the 25 instants. Architecture-wise, this block has 4 layers, two of them are LIF layers, while the other two are just fully connected layers whose purpose is to connect connect them. Here follows a table describing the parameters used:

FC1	`in_features=512,` `out_features=512`
LIF1	`beta=0.95,` `spike_grad=surrogate.fast_sigmoid()`
FC2	`in_features=768,` `out_features=768`
LIF2	`beta=0.95,` `spike_grad=surrogate.fast_sigmoid()`

Table 4: SNN Parameters

Figure 1 shows how the output of the linear projection plot is given as input to the SNN, in particular, it is fused with the output of LIF1 and then fed to FC2 (this is why `in_features` = 768 = 512 + 216, because 512 is given by the output of LIF1 that is the same of the BiGRU, while 216 is the length of the linear projection unit output). In order to enable the NN training with the backpropagation algorithm, the spikes must be approximated with smoother functions and this is exactly what `spike_grad=surrogate.fast_sigmoid()` does. Finally, the parameter `beta` is the decay factor, that controls how much of the membrane potential at the previous time instant influences the current potential, meaning that a higher `beta` makes the model remember more of its previous states. The value 0.95 has been chosen as the same as the one in the reference paper, however, since the SNN is the most critical unit of this project, this parameter has been subject to fine-tuning using the cross-validation technique to find the best value.

3.0.6 Decision Head Block

This is the last block of the proposed architecture, and it is just a three layers NN, as shown in the following table, whose purpose is to further elaborate the data making it 2-dimensional, to make it ready for the loss function that is going to handle the classification error for the two different classes.

Layer Type	Parameters
Linear	`in_features=768,` `out_features=512`
LeakyReLU	`negative_slope=0.1`
Dropout	`p=0.4`
Linear	`in_features=512,` `out_features=128`
LeakyReLU	`negative_slope=0.1`
Dropout	`p=0.4`
Linear	`in_features=128,` `out_features=2`

Table 5: Decision Head Block Parameters

4 Technologies Used

This section is a brief overview on the different technologies used to implement the model described previously, to train it and to test it.
The programming language chosen for the implementation is Python, and everything has been written on a Jupyter notebook running on Kaggle.com using the GPU P100 as hardware accelerator(except for the preprocessing phase that has been performed locally).
As regards the framework used to implement the NN, PyTorch was chosen to be coherent with the reference paper and also because it offers a clear interface to deal with SNN through the snntorch module, working perfectly with the other torch modules needed for the other blocks of the NN. Torch also offers other useful tools in the context of NN, like the `torch.utils.data` module, containing classes like `DataLoader` and `TensorDataset` that ease the syntax when writing training loops or the module `torchvision.transforms`, useful to transform images, a technique used in the training phase of the model as explained in the next section to promote generalization.
Other notable Pyhton modules that have been used are `numpy` and `matplotlib`, to deal with matrices and plots, and finally `sklearn` for confusion matrices, train/test splits and separation of the dataset in folds for the cross-validation.

5 Experimental Setup

The experiment can be divided into three main phases:

- **Preprocessing**
- **Cross-validation and Training**
- **Testing**

5.1 Preprocessing

Before actually training and testing the model, it is necessary to adequately prepare the input data in a form that the NN implemented with PyTorch can work with. From the "Chest X-Ray Images (Pneumonia)" dataset downloaded from Kaggle.com 5232 images have been used, of which 1349 depict sane patients while the other 3883 depict sick ones. The images, originally in .jpeg format, have been converted in numpy matrices and then in torch tensors using a train test split with `test_size`=0.2 keeping the proportion between the classes also in the the train and test set:

- Class 0 in train set: 1079
- Class 1 in train set: 3106
- Class 0 in test set: 270
- Class 1 in test set: 777

Where class 0 represents normal patients, while class 1 represents patients with pneumonia.

5.2 Cross-validation and Training

Before training the model on the dataset to test it, it is necessary to explore different choices for the architecture and for the hyperparameters to find the best performing NN. Since these choices cannot be directly done on the test set (otherwise we would overfit on the test data), the cross-validation technique has been used, in particular the 5-fold cross-validation. The way it works is by dividing the training set in five folds and, for each model to validate, iterating over them by training the NN on the training set, except for the fold of the current iteration, which is instead used for validation. So, each time cross-validation is performed, there are five values of validation loss that are averaged out. This is done to better generalize the performance of the model on unseen data without using too many samples.
Cross-validation has been used to choose among two different architectures, the model described in section 2 and then the same model but without the BiGRU block with five different values of the learning rate, that is [1e-2, 1e-3, 1e-4, 1e-5, 1e-6]. After this process, the best model is chosen and then submitted to another cross-validation to find the best value of beta (decay factor of the SNN) between the following: [0.5, 0.7, 0.8, 0.9, 0.95, 0.99].
After having selected the best combination of architecture and parameters, the newfound model has then been trained on the whole training set . In particular, both in the validation (in each fold) and in the training phase, the following parameters are set and the following steps are performed:

- **Hyperparameters**: `epochs` = 30, `batch_size` = 32, `weight_decay` = 1e-5;
- **Augmentation (Only in training because it made validation too long to execute)**: each time an image is extracted from the training set, there is a 70% probability that a random transformation is performed. In particular, the image can be randomly flipped, zoomed in, zoomed out, and brightened, to promote generalization in order to not overfit on the training data;
- **Optimization**: the optimization algorithm chosen is Adam with a OneCycleLR scheduler, meaning that the learning rate is decreased uniformly during all the epochs. Since the problem is a classification problem, the loss function chosen is the cross entropy loss function with a penalization on the loss for the pneumonia class to address the fact that the dataset is imbalanced.

5.3 Testing

The model found in the previous steps has then been trained on the test set. Moreover, to test its robustness, different kind of noises have been applied to the test set in order to see how the prediction changed. In particular, gaussian blur with sigma in [0.2, 0.4, 0.6], salt and pepper noise with probability to burn a pixel p in [0.01, 0.05, 0.1] and speckle noise with sigma in [0.1, 0.2, 0.3] have been used.

6 Experimental Results

Here follows a table with the results of the cross-validation for the model with the BiGRU unit:

Learning Rate	**Fold 1**	**Fold 2**	**Fold 3**	**Fold 4**	**Fold 5**	**Average Loss**
1e-2	0.2465	0.1996	0.2419	0.1673	0.6732	0.3057
1e-3	0.0772	0.1339	0.1982	0.2904	0.1334	0.1666
1e-4	0.1348	0.1125	0.1283	0.2076	0.1030	0.1372
1e-5	0.1329	0.0979	0.1366	0.1122	0.0851	**0.1130**
1e-6	0.3010	0.2381	0.2864	0.2722	0.2398	0.2675

Table 6: Validation losses

Here the results for the model without the BiGRU:

Learning Rate	**Fold 1**	**Fold 2**	**Fold 3**	**Fold 4**	**Fold 5**	**Average Loss**
1e-2	0.2829	0.1653	0.6656	0.2142	0.6724	0.4001
1e-3	0.1601	0.1446	0.1575	0.2269	0.1141	0.1606
1e-4	0.0945	0.1131	0.1326	0.1831	0.1134	0.1273
1e-5	0.0822	0.1158	0.1529	0.1382	0.1214	**0.1221**
1e-6	0.2869	0.2378	0.2611	0.2504	0.2643	0.2601

Table 7: Validation losses for the Model Without GRU

The results above show how the best model among the ones found above (the one with the lowest validation loss) is the full architecture with learning rate equal to 1e-5. However, unlike what the reference paper stated, the results are not that different. This could be due to the fact that the BiGRU is not that useful for static images, since it cannot extract anything for data lacking temporal dependencies. These instead are the results of the cross-validation for different values of `beta` for the best architecture with the best value of the learning rate, that is `lr` = 1e-5:

Beta (β)	**Fold 1**	**Fold 2**	**Fold 3**	**Fold 4**	**Fold 5**	**Average Loss**
0.50	0.1056	0.1067	0.1334	0.1362	0.1106	0.1185
0.70	0.1500	0.1128	0.1377	0.1292	0.1034	0.1266
0.80	0.1315	0.1074	0.1464	0.1381	0.1053	0.1257
0.90	0.1086	0.0975	0.1375	0.1487	0.0940	0.1172
0.95	0.1130	0.0966	0.1395	0.1079	0.1068	**0.1127**
0.99	0.1014	0.1218	0.1115	0.1391	0.0995	0.1147

Table 8: Validation losses for different values of the membrane potential decay factor (β)

The best value of `beta` is therefore 0.95, meaning that the most efficient value of beta is a high one, preserving the value of the membrane potential of past time instants.

The following are the results of the testing phase:

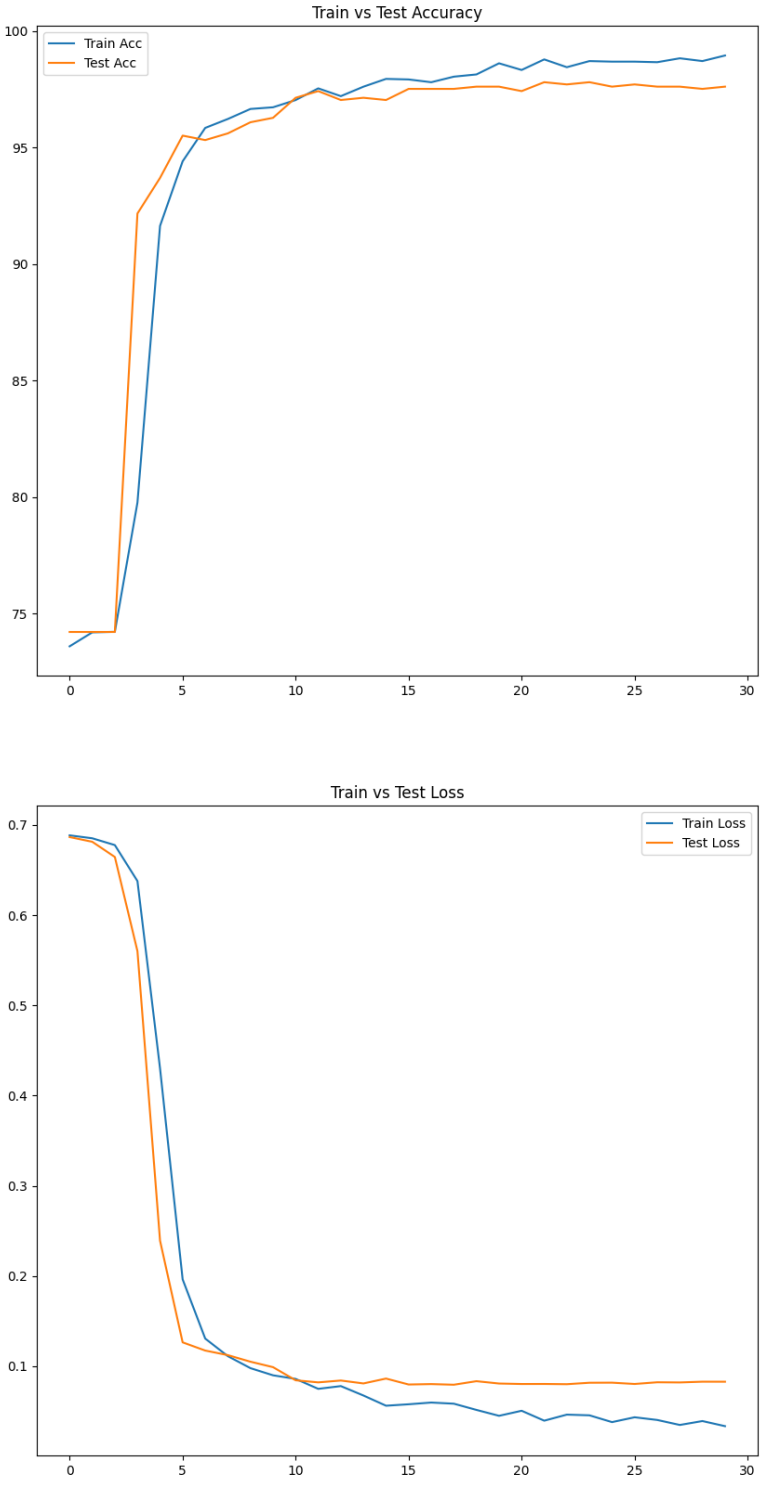


Figure 2: Accuracy and Loss for Training VS Testing

Note that the accuracy is displayed just for plotting purposes, since it is not that meaningful for imbalanced datasets. The plot of the loss shows how all the measures applied to contain the overfitting have worked since the gap between the two losses is not that large and the validation one plateaued.

Here a table describing all the testing metrics and the confusion matrix:

Metric	**Value**
Training Loss	0.0335
Test Loss	0.0828
Test Accuracy	97.61%
Precision	0.9845
Recall	0.9833
F1-Score	0.9839

Table 9: Performance metrics.

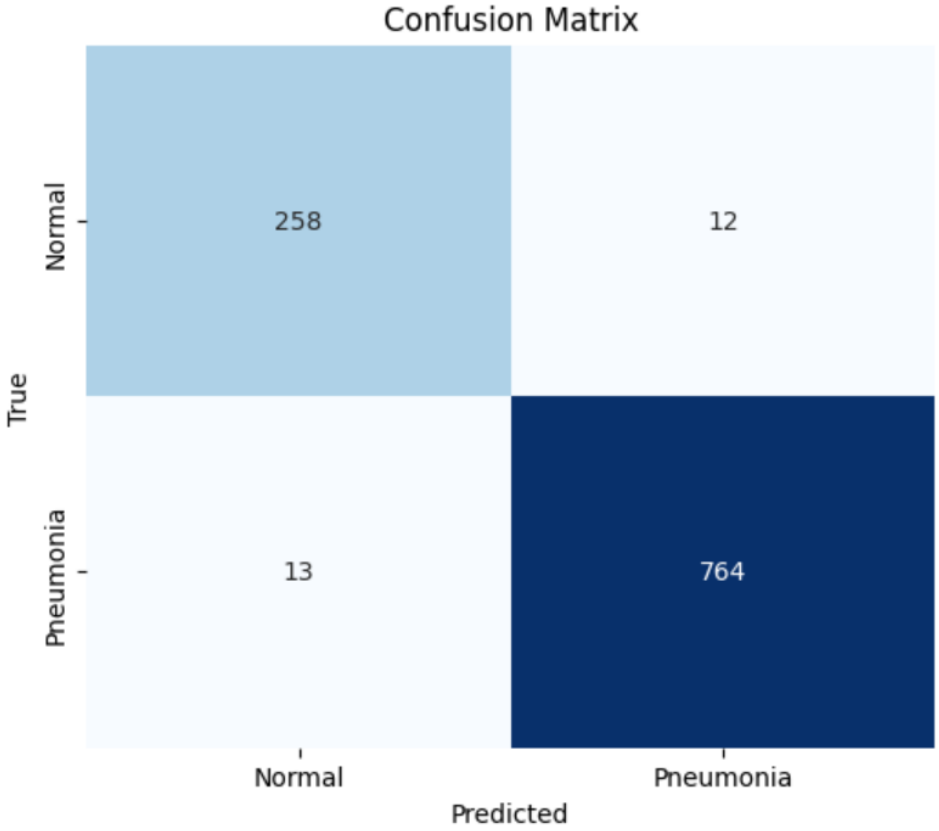


Figure 3: Confusion Matrix

All the performance metrics are high enough to say that the model behaves good on unseen data. In medical application, the most important metric is the recall, defined as the ratio between (True Positive) / (True Positive + False Negatives), is an index on how much unlikely it is for the model to say that the patient is sane while they could actually have pneumonia. The confusion matrix clearly shows the distribution of the different kinds of prediction, where a limitation of this model can be seen: despite the classes being unbalanced, the false negatives are almost equal to the false negatives, while a better model would have kept the proportion also in the wrong predictions.

The following images graphically show how the different kinds of noise affect the original image and how, for a single image, the prediction changes.

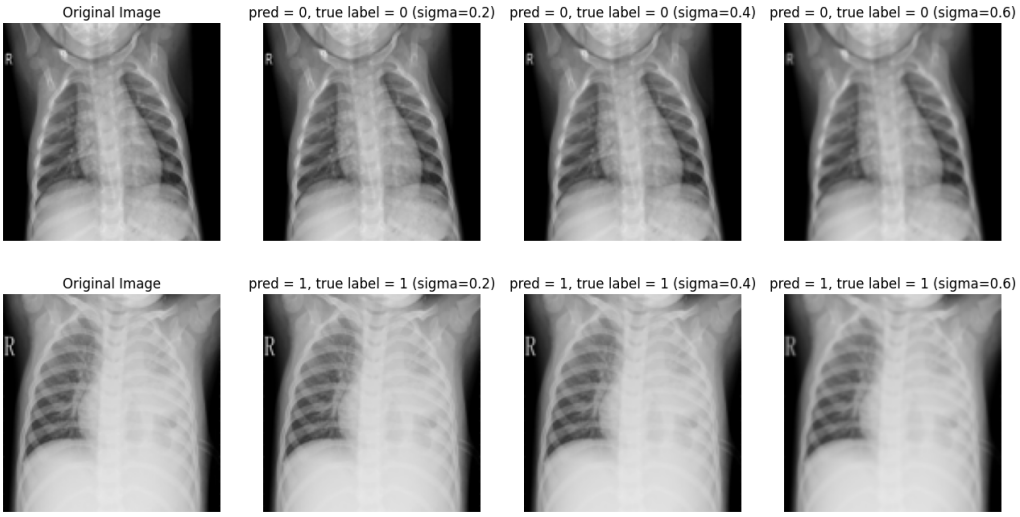


Figure 4: Gaussian Blur

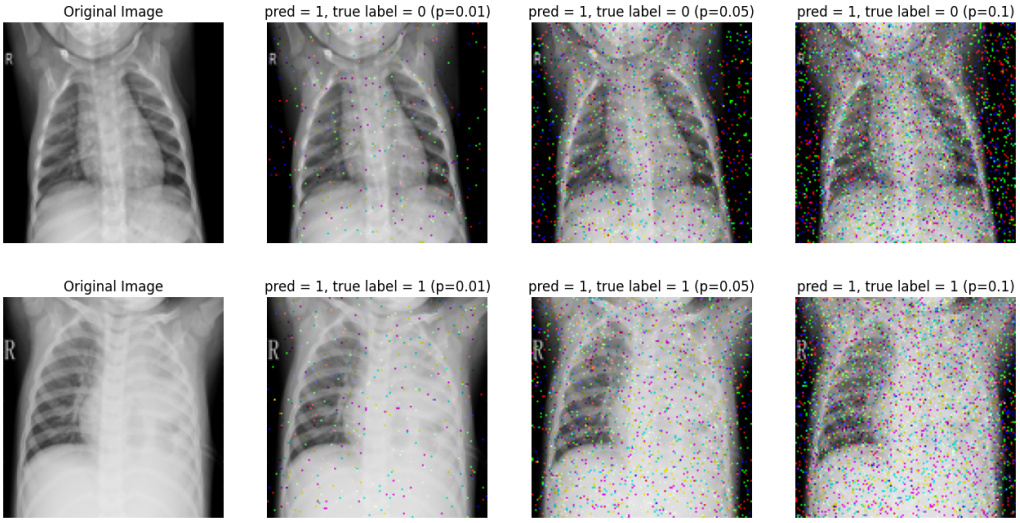


Figure 5: Salt and Pepper Noise

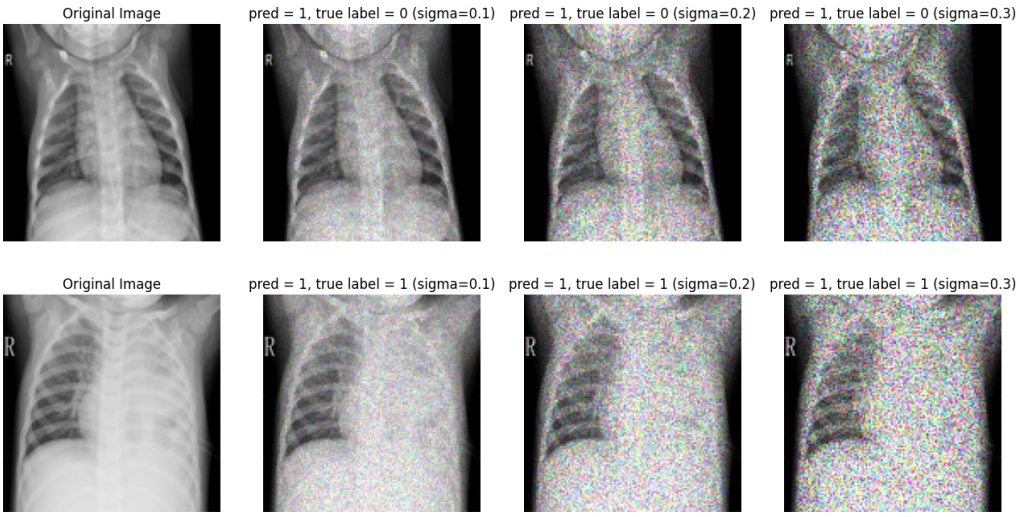


Figure 6: Speckle Noise

The following tables show the result of using the model on the whole test set modified with the three different noises.

Blur Intensity (σ)	**Discrepancies (Count)**	**Discrepancy Rate (%)**
$\sigma = 0.2$	0	0.00%
$\sigma = 0.4$	0	0.00%
$\sigma = 0.6$	13	1.24%

Table 10: Robustness to Gaussian Blur

The model is very robust with respect to gaussian blur since discrepancies are detected only for the highest value of σ, and for a very small percentage. This is a very good sign, because blurring is the most common noise in real life and the model does not seem affected by it.

Noise Density (p)	**Discrepancies (Count)**	**Discrepancy Rate (%)**
p = 0.01	184	17.57%
p = 0.05	270	25.79%
p = 0.1	271	25.88%

Table 11: Robustness to Salt and Pepper Noise

Noise Variance (σ)	**Discrepancies (Count)**	**Discrepancy Rate (%)**
$\sigma = 0.1$	118	11.27%
$\sigma = 0.2$	270	25.79%
$\sigma = 0.3$	291	27.79%

Table 12: Robustness to Speckle Noise

With both the salt & pepper and the speckle noise the model performs poorly because on average it changes its prediction on every four samples. This could be due to the fact that the sudden changes in pixels generated by these noises may trigger some spikes in the SNN that

lead to a different prediction. However, these results are not that concerning because in real life such a damaged image would never be used for a diagnosis unlike a slightly blurred image.

7 Conclusion

This study evaluated the performance of a Spiking Neural Network (SNN) architecture for the automated classification of pneumonia from chest X-ray images. The experimental results demonstrate that the proposed model achieves high diagnostic accuracy.
The ablation study compared the full architecture with a simplified version lacking the BiGRU layer. While the full model yielded the best overall validation loss, the performance gap was marginal. This suggests that for static imagery, the recurrent block primarily processes redundant temporal information, providing limited added value compared to the spatial features extracted by the CNN backbone.
In conclusion, this work confirms that Spiking Neural Networks are a viable and high-performing alternative for medical image classification, offering a robust framework that balances sensitivity and precision while maintaining a biologically inspired computational approach.